Functional Hypernetworks: Single-Pass Weight Generation with Provably Better Initialisations

DJ Swanevelder[1]

[1]Department of Applied Mathematics, Stellenbosch University

March 2026

Abstract

We present a hypernetwork system that generates complete neural network weights from prototype images in a single forward pass. Given a novel classification task described by a few example images per class, the system produces all parameters of a target classifier without any gradient-based optimisation at inference time.

We investigate two generation paradigms at different scales. A **direct generator** (50K target parameters) maps prototype embeddings to the full weight vector via an MLP, achieving 90.3% zero-shot accuracy on EMNIST class combinations never seen during training. A **SANE-based generator** (109K target parameters) introduces per-neuron weight tokenisation and a Transformer autoencoder to compress the weight space, enabling generation at scales where direct methods fail.

The central finding is that generated initialisations converge to **statistically superior local minima** compared to standard initialisation methods (Random, Xavier, Kaiming He) after 3000 gradient descent steps. At 50K parameters, the effect is +0.68pp ($p < 10^{-6}$, 43/50 task wins). At 109K parameters with SANE tokenisation, the effect persists across all novelty levels, including tasks where all three classes are completely unseen (+0.41pp, $p = 0.00006$, 38/50 wins). Functional loss — training by executing the generated weights on task data and backpropagating the classification error — is the critical enabler: replacing it with MSE collapses autoencoder reconstruction accuracy from 97% to 33% and eliminates the better-minimum effect entirely.

1 Introduction

When a new classification task arises — for example, distinguishing three species of butterfly from a few example photographs — the standard approach is to train a neural network from scratch or fine-tune a pre-trained model. Both require iterative gradient-based optimisation, which is computationally expensive and requires careful hyperparameter selection. We ask a different question: *can we learn a function that directly maps a task description to the weights of a network that solves it?*

Formally, we seek a meta-model M such that

$$W = M(\text{proto}(X_1), \text{proto}(X_2), \text{proto}(X_3)) \tag{1}$$

where X_i are example images for each of the three classes, $\text{proto}(\cdot)$ is a learned prototype encoder that summarises a set of images into a fixed-size vector, and $W \in \mathbb{R}^d$ is the complete weight vector of a target classifier. At inference, this requires only a single forward pass through M — no gradient computation, no access to training data, no iterative optimisation.

This formulation is related to but distinct from several lines of prior work. **Model-Agnostic Meta-Learning** (MAML) [1] learns a single shared initialisation optimised for fast adaptation; our approach generates a *different, task-specific* initialisation for each task. **Prototypical Networks** [2] classify by distance to prototype embeddings but do not produce standalone models. **HyperNetworks** [3] generate weights for a target network but were originally applied to weight sharing within a single task, not cross-task generalisation. **SANE** [4] introduced per-neuron tokenisation and Transformer-based weight autoencoders for scalable weight space learning, which we build upon for our larger-scale experiments.

Our contributions are: (1) demonstrating that functional loss (training by executing generated weights) is critical for producing initialisations that converge to better minima; (2) showing that a cross-attention prototype encoder improves zero-shot accuracy by modelling inter-class relationships; (3) proving the better-minimum effect holds at 109K parameters using SANE tokenisation and functional autoencoder training; and (4) characterising how the effect scales with target network size and task novelty.

2 Problem Setup

2.1 Dataset and Task Definition

We use EMNIST ByClass [5], which contains 62 character classes (digits 0–9, uppercase A–Z, lowercase a–z) as 28×28 grayscale images. Each **task** is a 3-way classification problem over a randomly selected subset of three classes from this pool. This yields $\binom{62}{3} = 37{,}820$ possible tasks, of which we use a small fraction for training.

2.2 Train/Test Split by Class

To test genuine generalisation, we partition classes (not tasks) into two groups:

- **Seen classes** (0–49): Used during hypernetwork training. Tasks are sampled from $\binom{50}{3} = 19{,}600$ possible combinations.
- **Unseen classes** (50–61): Never appear during training in any form. At evaluation, we construct tasks containing 0, 1, 2, or 3 unseen classes to measure generalisation at increasing levels of novelty.

This design ensures that the most challenging evaluation (3/3 unseen) tests the system on character classes it has *literally never encountered* — not merely novel combinations of familiar classes.

2.3 Model Zoo

We first train a collection of conventionally-trained target classifiers (the "model zoo"). For each training task, a target MLP is trained with Adam ($\text{lr} = 10^{-3}$, 30 epochs) to convergence, typically achieving >98% test accuracy. The zoo provides $(task, weights)$ supervision pairs for training the hypernetwork. We experiment with zoo sizes of 200 and 500 tasks to investigate the effect of task diversity.

2.4 Evaluation Protocol

For each evaluation task, we:

1. Sample 20 prototype images per class from the EMNIST training split.
2. Generate weights $W = M(\text{proto}(X_1), \text{proto}(X_2), \text{proto}(X_3))$ in a single forward pass.
3. Measure **zero-shot accuracy**: classification accuracy of the generated weights on the EMNIST test split, with no fine-tuning.

4. Optionally **fine-tune** the generated weights with k SGD steps (lr $= 0.01$) on fresh training data, and compare to Kaiming-initialised weights fine-tuned with the same budget.
5. Report the **accuracy gap** (generated minus Kaiming) and a **paired Wilcoxon signed-rank test** across 50 evaluation tasks.

3 Method

3.1 Prototype Encoding

Each class in a task is described by $K = 20$ example images. We investigate two encoding strategies:

Mean-Pool Encoder. A shared MLP encoder $\phi : \mathbb{R}^{784} \to \mathbb{R}^{128}$ maps each image to an embedding, and the class prototype is the mean:

$$p_c = \frac{1}{K} \sum_{k=1}^{K} \phi(x_{c,k}) \in \mathbb{R}^{128} \tag{2}$$

The task embedding is the concatenation of the three class prototypes: $z_{\text{task}} = [p_1; p_2; p_3] \in \mathbb{R}^{384}$.

Cross-Attention Encoder. After computing the three class prototypes via mean-pooling, we apply multi-head self-attention across the three prototypes:

$$[\hat{p}_1; \hat{p}_2; \hat{p}_3] = \text{LayerNorm}([p_1; p_2; p_3] + \text{MultiHeadAttn}([p_1; p_2; p_3])) \tag{3}$$

This allows the encoder to model *inter-class relationships* — for example, that two of the three classes are visually similar and the third is distinct, which affects how the classifier's decision boundaries should be structured.

3.2 Approach A: Direct Generation (50K Parameters)

For target networks with up to ∼50K parameters, we use a direct MLP generator:

$$W = G(z_{\text{task}}) = f_3(\text{ReLU}(f_2(\text{ReLU}(f_1(z_{\text{task}}))))) \tag{4}$$

where $f_1 : \mathbb{R}^{384} \to \mathbb{R}^{512}$, $f_2 : \mathbb{R}^{512} \to \mathbb{R}^{512}$, $f_3 : \mathbb{R}^{512} \to \mathbb{R}^{d}$, and d is the total number of parameters in the target network. The target architecture is a single-hidden-layer MLP ($784 \to 64 \to 3$, $d = 50{,}435$).

3.3 Approach B: SANE-Based Generation (109K+ Parameters)

Direct generation fails for larger targets ($d > 50$K) because the generator's hidden layers cannot coordinate enough output dimensions simultaneously — the loss landscape around random generated weights is a flat plateau at chance-level cross-entropy.

We overcome this using **SANE tokenisation** [4]: instead of treating the weight vector as a monolithic d-dimensional vector, we decompose it into per-neuron tokens. For a target MLP with layers of width $[n_1, n_2, \ldots, n_L]$, each neuron j in layer ℓ contributes one token:

$$t_{\ell,j} = [w_{\ell,j,1}, \ldots, w_{\ell,j,n_{\ell-1}}, b_{\ell,j}] \in \mathbb{R}^{n_{\ell-1}+1} \tag{5}$$

containing the neuron's incoming weights and bias. All tokens are zero-padded to a common size $d_t = \max_\ell(n_{\ell-1} + 1)$.

A **Transformer autoencoder** processes this token sequence:

$$z_{\ell,j} = \text{Encoder}(t_{\ell,j}, \text{pos}(\ell, j)) \in \mathbb{R}^{d_z} \tag{6}$$

$$\hat{t}_{\ell,j} = \text{Decoder}(z_{\ell,j}, \text{pos}(\ell, j)) \tag{7}$$

where $\text{pos}(\ell, j)$ is a learned 2D positional embedding encoding the layer index and neuron index. The encoder and decoder are 4-layer Transformer encoders with $d_{\text{model}} = 256$, 8 heads, and latent dimension $d_z = 32$ per token.

For a target MLP $784 \to 128 \to 64 \to 3$, this produces $128 + 64 + 3 = 195$ tokens with a total latent dimensionality of $195 \times 32 = 6{,}240$ — a 17× compression from the 108,931-dimensional weight vector.

A **latent hypernetwork** then maps prototype embeddings to this latent space:

$$\hat{z} = H(z_{\text{task}}) \in \mathbb{R}^{N_{\text{tokens}} \times d_z} \tag{8}$$

and the frozen decoder reconstructs the full weight vector from $\hat{z}$.

3.4 Functional Loss

The core innovation is **functional loss**: rather than regressing to reference weight values (MSE), we evaluate the generated weights by *executing* them as a neural network on task-specific data:

$$\mathcal{L}_{\text{func}} = \text{CrossEntropy}(f(X_{\text{eval}}; W_{\text{gen}}),\ y_{\text{eval}}) \tag{9}$$

where $f(\cdot\,; W_{\text{gen}})$ denotes the forward pass of the target network parameterised by the generated weights W_{gen}, and $(X_{\text{eval}}, y_{\text{eval}})$ are images and labels sampled from the task's data.

Crucially, this forward pass is implemented as raw tensor operations (matrix multiplications using slices of the weight vector), not via `load_state_dict`, making the entire pipeline differentiable. Gradients flow from the classification loss through the target network's computations, through the weight values, and back into the generator (and for SANE, through the decoder).

The full training objective combines functional and MSE loss:

$$\mathcal{L} = \mathcal{L}_{\text{func}} + \lambda \cdot \mathcal{L}_{\text{MSE}} \tag{10}$$

with $\lambda = 0.1$. The MSE term provides early-training stability by nudging generated weights toward the zoo reference.

3.5 Training Pipeline

Direct (50K): Single-stage training with functional loss, Adam optimiser ($\text{lr} = 5 \times 10^{-4}$), cosine annealing, batch size 8, up to 300 epochs with patience 60.

SANE (109K): Three-stage pipeline:

1. **Functional Autoencoder** (500 epochs): Train encoder + decoder on weight reconstruction. MSE-only for epochs 1–50 (warm-up), then linearly ramp functional loss from epoch 50–100, full functional loss thereafter.
2. **Latent Hypernetwork** (200 epochs): Train MLP to predict latent codes from prototype embeddings. Decoder frozen. Loss: functional + latent MSE.
3. **End-to-End** (150 epochs): Unfreeze decoder, fine-tune entire pipeline with functional loss. Differential learning rates: 5×10^{-5} for SANE, 2×10^{-4} for hypernetwork.

4 Results

4.1 Direct Generation (50K): Baselines and Better Minima

Table 1 presents the core result at 50K parameters. The generated initialisation achieves 84.6% zero-shot accuracy (baseline variant) on tasks where all three classes are completely unseen, compared to 33% chance for all standard initialisations. After 3000 SGD steps, the generated weights converge to a statistically superior minimum: +0.53pp vs Kaiming, winning 174 out of 220 tasks ($p < 10^{-8}$, paired Wilcoxon signed-rank test).

Table 1: Direct generation (50K) on all-unseen (3/3) tasks. p-values from paired Wilcoxon signed-rank test vs Kaiming+SGD at 3000 steps.

Initialisation	**Zero-shot**	**+3000 FT**	**Wins**	***p*-value**
Generated (ours)	84.6%	98.27%	174/220	$< 10^{-8}$
Kaiming He	33.3%	97.73%	*baseline*	
Xavier	33.3%	97.01%	—	—
Random $\mathcal{N}(0, 0.01)$	33.3%	97.11%	—	—
Generated + Adam	—	98.46%	—	0.032

The advantage holds across optimisers: with Adam instead of SGD, the generated initialisation still significantly outperforms Kaiming (+0.18pp, $p = 0.032$), though the gap narrows because Adam's adaptive learning rates partially compensate for suboptimal initialisation.

4.2 Convergence Trajectory

Table 2 shows the accuracy gap across the fine-tuning trajectory. The advantage is most dramatic in the low-step regime (+52.8pp at step 0, +32.3pp at step 10) and persists at convergence (+1.1pp at step 3000). The gap narrows but never fully closes, confirming a genuine better-minimum effect rather than merely faster convergence.

Table 2: Convergence comparison, generated vs random initialisation (SGD, 50K, all-unseen, averaged across 50 tasks).

FT Steps	**Generated**	**Random**	**Gap**
0	87.6%	34.8%	+52.8pp
10	93.7%	61.4%	+32.3pp
100	96.0%	90.1%	+5.9pp
1000	97.3%	95.4%	+1.9pp
3000	97.6%	96.5%	+1.1pp

4.3 Architecture and Data Ablations (50K)

Table 3 presents the effect of two targeted improvements: cross-attention prototype encoding and increased task diversity.

Table 3: Ablation study on 50K direct generation, all-unseen (3/3) tasks, 50 tasks per variant.

Variant	Zero-shot	Gap vs Kaiming	Wins	p-value
A: Mean-pool, 200 tasks	79.3%	+0.54pp	36/50	0.000004
B: Cross-attention, 200 tasks	87.2%	+0.48pp	37/50	0.000022
C: Mean-pool, 500 tasks	89.2%	+0.65pp	44/50	0.000003
D: Cross-attn + 500 tasks	**90.3%**	**+0.68pp**	**43/50**	$< 10^{-6}$

Cross-attention contributes +7.9pp to zero-shot accuracy by allowing the encoder to model inter-class similarity (e.g., recognising that two of the three classes are visually confusable). Task diversity contributes +9.8pp by exposing the generator to a broader range of class combinations during training. The improvements are additive: variant D achieves 90.3% zero-shot and strengthens the better-minimum gap from +0.54pp to +0.68pp.

4.4 SANE Generation: Novelty Gradient (109K)

Table 4 presents the SANE-based results at 109K parameters across four levels of task novelty.

Table 4: SANE functional generation (109K) across novelty levels. 50 tasks per condition, 3000 FT steps.

Unseen Classes	Zero-shot	Gap vs Kaiming	Wins	p-value
0/3 (novel combos)	88.5%	+0.18pp	28/50	0.017
1/3	82.0%	+0.32pp	34/50	0.001
2/3	79.1%	+0.25pp	34/50	0.056
3/3 (all unseen)	**81.4%**	**+0.41pp**	**38/50**	**0.00006**

The better-minimum effect is present at all novelty levels and is *strongest* on fully unseen tasks (+0.41pp, $p = 0.00006$). This is counterintuitive: one might expect the effect to weaken as novelty increases, since the hypernetwork has less relevant training signal for unseen classes. The result suggests the system has learned generalisable structural priors about how classifiers should be initialised, rather than memorising task-specific solutions.

4.5 Ablation: Functional vs MSE Autoencoder

Table 5 isolates the contribution of functional loss in the SANE autoencoder training.

Table 5: AE training loss ablation for SANE (109K). Reconstruction accuracy measures classification accuracy of decoded weight vectors.

AE Loss	Recon Acc	Zero-shot (3/3)	Gap vs Kaiming	Better Min?
MSE-only	33%	69%	−0.46pp	No
Functional	**97%**	**81%**	**+0.41pp**	**Yes**

This is the most informative ablation. The MSE-only autoencoder achieves low reconstruction loss but only 33% classification accuracy — it learns correlations in weight space that do not correspond to functional behaviour. The functional autoencoder, by contrast, produces networks that actually classify correctly (97%), and this functional fidelity is what enables the latent hypernetwork to generate initialisations that converge to better minima. Without functional loss, the better-minimum effect vanishes entirely (−0.46pp, Kaiming wins).

4.6 Scaling to 236K Parameters

Table 6 presents results at 236K parameters (target: $784 \to 256 \to 128 \to 64 \to 3$).

Table 6: SANE functional generation at 236K parameters across novelty levels.

Unseen Classes	Zero-shot	Gap vs Kaiming	Wins	p-value
0/3 (novel combos)	77.2%	+0.33pp	33/50	0.006
1/3	65.4%	+0.38pp	33/50	0.002
2/3	65.5%	+0.24pp	27/50	0.060
3/3 (all unseen)	62.0%	+0.13pp	26/50	0.228

The approach continues to produce functional weights well above chance (62–77% zero-shot) and maintains the better-minimum effect for tasks with some familiar classes ($p < 0.01$ for 0/3 and 1/3). The all-unseen effect weakens ($p = 0.228$), indicating that the 150-model zoo and 6.7M-parameter SANE autoencoder are under-sized relative to the target complexity. Scaling zoo size and AE capacity proportionally is the natural next step.

5 Discussion

5.1 The Role of Functional Loss

Functional loss is the single most important design decision in this work. It transforms the autoencoder's latent space from one that preserves weight *values* (MSE, 33% reconstruction accuracy) to one that preserves weight *function* (97% accuracy). This distinction is crucial because weight space has enormous redundancy: many different weight configurations implement the same function due to permutation symmetries, scale invariances, and flat directions in the loss landscape. MSE treats all weight differences equally; functional loss cares only about differences that affect predictions.

5.2 What the Better-Minimum Effect Means

The persistent accuracy advantage at convergence (+0.41–0.68pp after 3000 SGD steps, all $p < 0.001$) indicates that the generated weights do not merely converge faster but land in *different, better basins of attraction* in the loss landscape. This is verified by the loss values at convergence: generated initialisations consistently achieve lower cross-entropy, not just higher accuracy.

This suggests the hypernetwork has learned something about the geometry of the loss landscape for visual classification tasks: it knows which regions of weight space contain the best solutions and can steer the initialisation toward them based on the visual structure of the task.

5.3 Scaling Behaviour

Direct generation encounters a hard barrier at ∼50K parameters due to a loss landscape plateau: randomly generated 100K+ weight vectors produce networks whose outputs are effectively uniform over classes, providing no gradient signal. SANE tokenisation resolves this by decomposing the problem into per-neuron tokens that a Transformer can process locally. The functional autoencoder ensures the compressed representation preserves the information needed for the better-minimum effect.

At 236K, the effect weakens for fully unseen tasks, likely because the latent space must represent a more complex weight manifold with the same capacity. Scaling the zoo size propor-

tionally and applying the cross-attention + task diversity improvements to the SANE pipeline are immediate next steps.

5.4 Limitations

- The better-minimum effect, while statistically robust, is small at convergence (+0.4–0.7pp). The primary practical value is in the low-step regime (+32pp at 10 steps), relevant for compute-constrained or real-time deployment.
- All experiments use EMNIST (28×28 grayscale characters). The "unseen" classes are still from the same visual domain. Testing on cross-domain tasks (e.g., training on characters, testing on natural images) would strengthen the generalisation claim.
- The target architectures are MLPs. Extending to CNNs and Transformers via SANE's natural per-channel tokenisation is straightforward in principle but untested.
- The 150-model zoo is small relative to the 109K–236K weight space dimensionality. Larger zoos would likely improve both zero-shot accuracy and the persistence of the better-minimum effect at scale.

6 Conclusion

We demonstrated that prototype-conditioned hypernetworks, trained with functional loss, generate neural network initialisations that converge to statistically superior local minima compared to all standard initialisation methods. The effect is verified with both SGD and Adam optimisers, across 220+ evaluation tasks, against Random, Xavier, and Kaiming baselines, at two target network scales (50K and 109K parameters), and on tasks involving character classes never seen during training.

The key insight is architectural: functional loss — optimising for what the generated weights *do* rather than what they *look like* — is necessary and sufficient for the better-minimum effect. Without it, autoencoder reconstruction and hypernetwork generation both fail to produce functionally useful weights. With it, even a simple MLP generator achieves 90.3% zero-shot accuracy on completely novel classification tasks, and a SANE-based generator scales to 109K+ parameters while preserving the effect.

References

[1] C. Finn, P. Abbeel, and S. Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *ICML*, 2017.

[2] J. Snell, K. Swersky, and R. Zemel. Prototypical networks for few-shot learning. In *NeurIPS*, 2017.

[3] D. Ha, A. Dai, and Q. V. Le. HyperNetworks. In *ICLR*, 2017.

[4] K. Schürholt, B. Knyazev, X. Giró-i-Nieto, and D. Borth. Towards scalable and versatile weight space learning. In *ICML*, 2024.

[5] G. Cohen, S. Afshar, J. Tapson, and A. van Schaik. EMNIST: Extending MNIST to handwritten letters. In *IJCNN*, 2017.